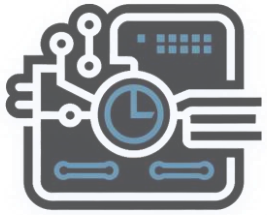


Progettazione Elettronica Digitale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

L05 – Sommario

➔Sintesi di FSM elementari
➔Flusso di progetto



Macchine a stati finiti (finite-state machine – FSM)
Definizione

Cos'è una FSM?

- Un sistema digitale che memorizza il proprio «stato» e reagisce agli ingressi cambiando stato

Componenti fondamentali

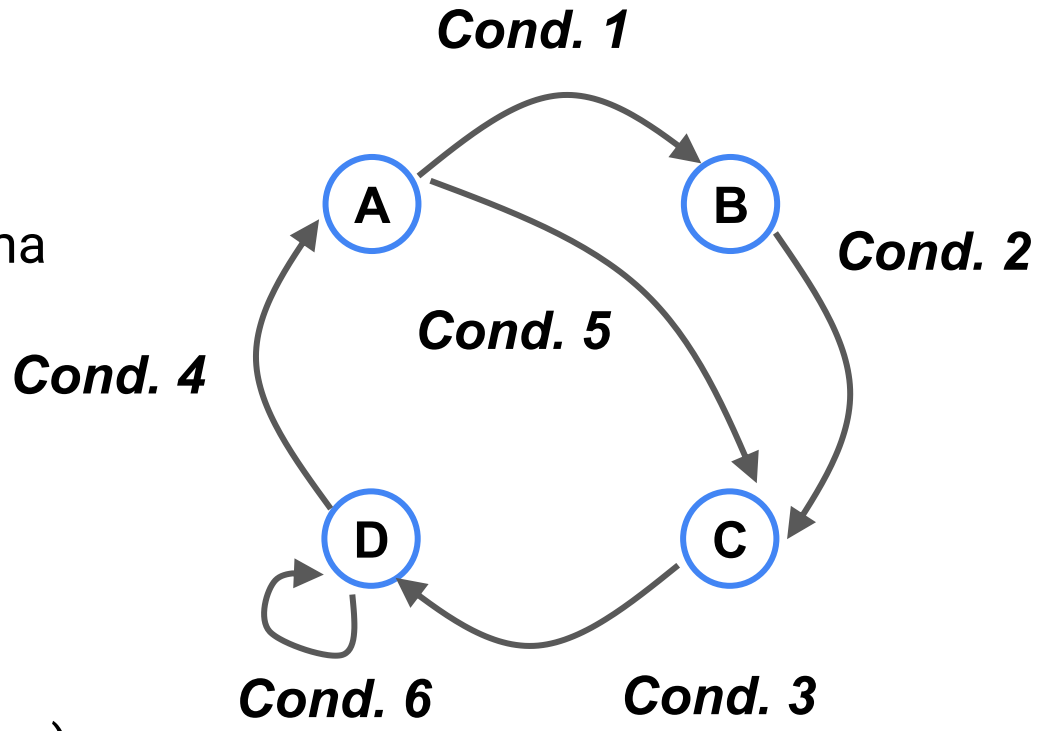
- *Elementi di memoria* (flip-flop)
  - La loro condizione (0/1) identifica lo *stato corrente* del sistema
  - Determinano la «memoria» del sistema
- *Stati discreti*
  - Insieme finito di *configurazioni possibili*
  - Ogni stato rappresenta una *condizione specifica* del sistema
- *Comportamento sequenziale*
  - Le *uscite* e lo *stato futuro* dipendono dallo *stato presente*
  - Due modelli principali: *Moore* e *Mealy* (approfonditi più avanti)

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

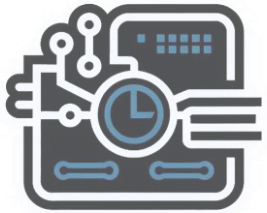
Macchine a stati finiti (finite-state machine – FSM)
Diagramma degli stati

Rappresentazione grafica di una FSM

- *Stati* → rappresentati da cerchi/nodi
  - Ogni *cerchio* identifica una *configurazione* del sistema
  - Contengono l'etichetta dello stato (A, B, C, D, …)
- *Transizioni* → rappresentate da *archi/frecce*
  - I *passaggi* da stato a stato sono *condizionati da variabili* di ingresso e/o interne
  - Le *condizioni di uscita* devono essere *mutuamente esclusive* (per ogni combinazione di ingressi, un *solo* arco attivo)
- *Uscite* → associate a *stati* (Moore) o *transizioni* (Mealy)
  - Ad ogni *stato o transizione* corrisponde una specifica *combinazione delle uscite*



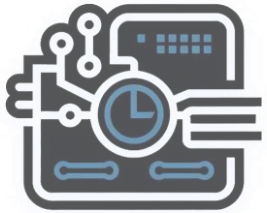
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Flusso di progetto di una FSM

- È articolato in una serie di fasi successive
  - Analisi delle specifiche
  - Definizione astratta della FSM
  - Minimizzazione degli stati
  - Assegnazione degli stati
  - Scelte tecnologiche e implementative
  - Sintesi della FSM
- Vedremo in seguito alcuni esempi completi del flusso di progetto
- Poi si approfondiranno più in dettaglio i passi legati all'*ottimizzazione* e *implementazione* delle FSM

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

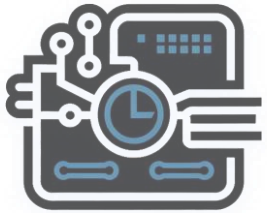


Esempi di progettazione FSM

- Verranno presentati tre esempi di design FSM di complessità crescente

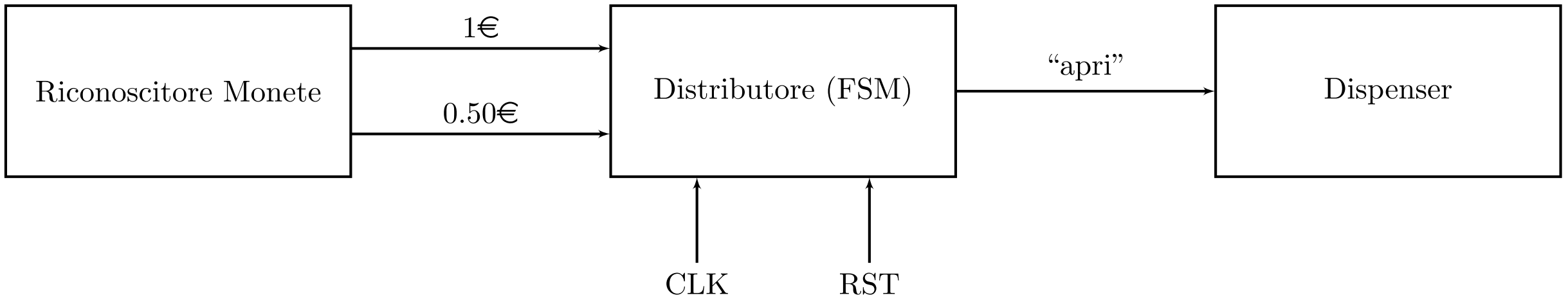
1. Distributore automatico
   - Applicazione del *flusso di progetto completo*
   - FSM con *pochi stati* e *transizioni semplici*
   - *Introduzione* alla metodologia
2. Riconoscitore di sequenza «1101»
   - Gestione di *sottosequenze sovrapposte*
   - Confronto tra architetture *Mealy* e *Moore*
   - Maggiore *complessità nelle transizioni*
3. Controllore semaforico
   - Sistema con *temporizzatori esterni*
   - Gestione di *segnali multipli* (sensori + timer)
   - Esempio di FSM in *sistemi real-time*

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 1: distributore automatico

- Descrizione informale: un distributore automatico accetta solo monete da 1€ o 50¢, eroga il prodotto al raggiungimento dell'importo di 1,50€ e *non* dà resto
- Passo 1: *analisi* del problema
  - Si indentificano gli *elementi* che compongono il sistema
  - Si identificano i *segnali* e gli *stimoli* a cui il sistema deve *reagire*
  - Si identificano i *comportamenti attesi* del sistema
  - Si tenta di *schematizzare* il sistema

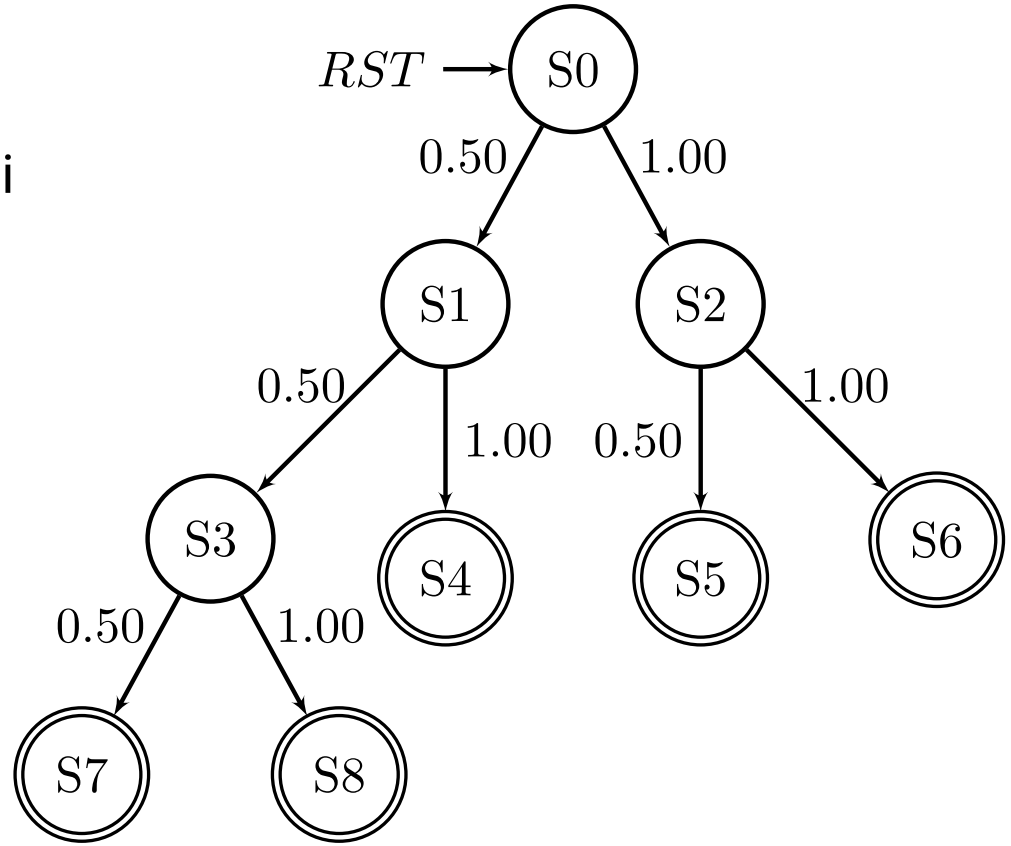


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1: distributore automatico

A1

- Passo 2: *formalizzazione* della FSM e delle sue *transizioni*
  - Si definisce lo *stato iniziale*
  - Si considerano i *valori* che possono assumere gli ingressi, stato per stato e si definisce l'*evoluzione* del sistema
  - Si identificano gli eventuali *stati terminali* e il *valore delle uscite* associato a ogni stato

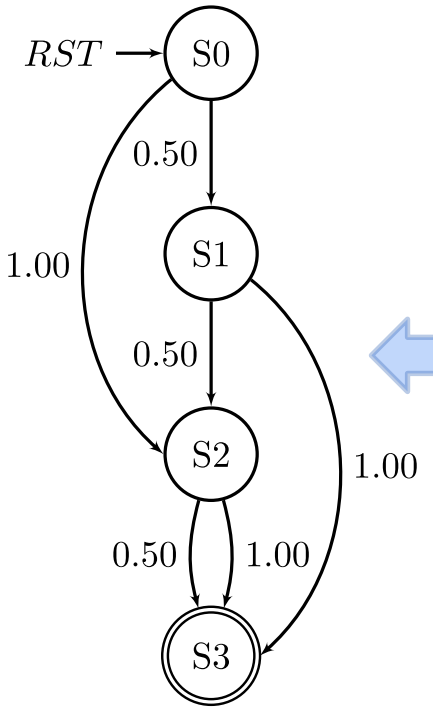


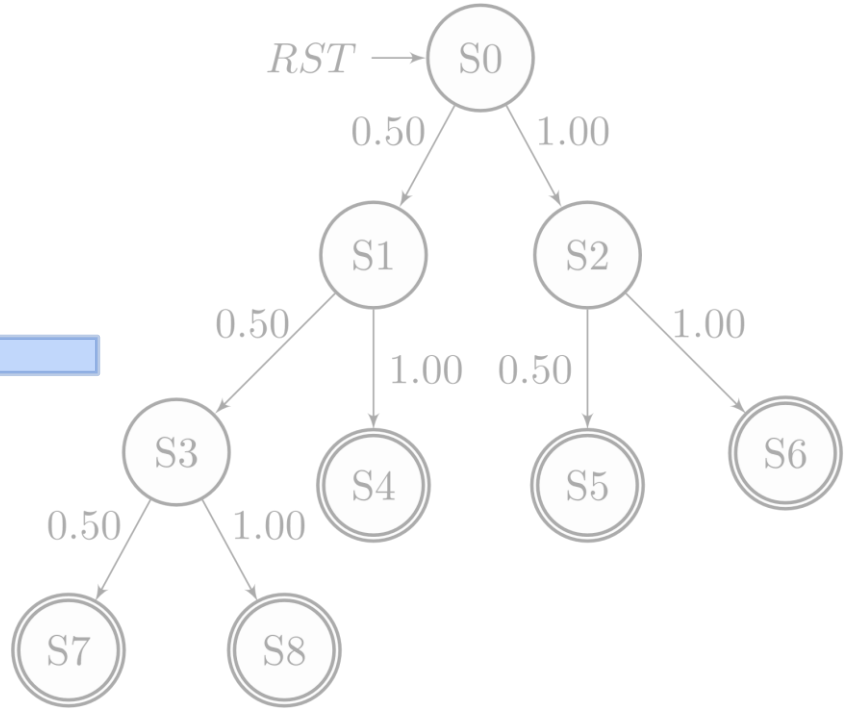
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1: distributore automatico

- Passo 3: *minimizzazione* degli stati (approfondito nella lezione successiva)
  - Si indentificano gli *stati equivalenti* per ridurne il numero complessivo
    - Si riutilizzano gli stati esistenti, quando possibile
  - Si definisce una *tabella simbolica degli stati* per visualizzare in maniera alternativa il comportamento della macchina

Stato Presente	1.00	0.50	Stato Futuro	"apri"
0.00	0	0	0.00	0
	0	1	0.50	0
	1	0	1.00	0
	1	1	X	X
0.50	0	0	0.50	0
	0	1	1.00	0
	1	0	1.50	0
	1	1	X	X
1.00	0	0	1.00	0
	0	1	1.50	0
	1	0	1.50	0
	1	1	X	X
1.50	X	X	1.50	1





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1: distributore automatico

- Passo 4: *codifica degli stati* (approfondita nella lezione successiva)
  - Si sceglie una *rappresentazione adatta* per codificare gli stati
  - Si passa dalla *tabella simbolica* alla *tabella codifica degli stati*
    - Diamo nomi simbolici a ingressi e uscite

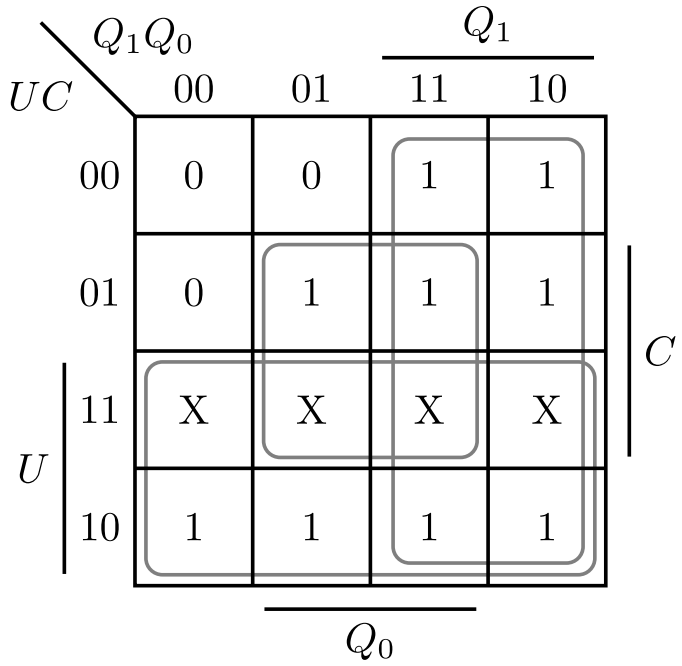
Stato Presente	1.00	0.50	Stato Futuro	"apri"
0.00	0	0	0.00	0
	0	1	0.50	0
	1	0	1.00	0
	1	1	X	X
0.50	0	0	0.50	0
	0	1	1.00	0
	1	0	1.50	0
	1	1	X	X
1.00	0	0	1.00	0
	0	1	1.50	0
	1	0	1.50	0
	1	1	X	X
1.50	X	X	1.50	1

Stato Presente		Ingressi		Stato Futuro		Uscite
Q_1	Q_0	U	C	D_1	D_0	O
0	0	0	0	0	0	0
		0	1	0	1	0
		1	0	1	0	0
		1	1	X	X	X
0	1	0	0	0	1	0
		0	1	1	0	0
		1	0	1	1	0
		1	1	X	X	X
1	0	0	0	1	0	0
		0	1	1	1	0
		1	0	1	1	0
		1	1	X	X	X
1	1	0	0	1	1	1
		0	1	1	1	1
		1	0	1	1	1
		1	1	X	X	X

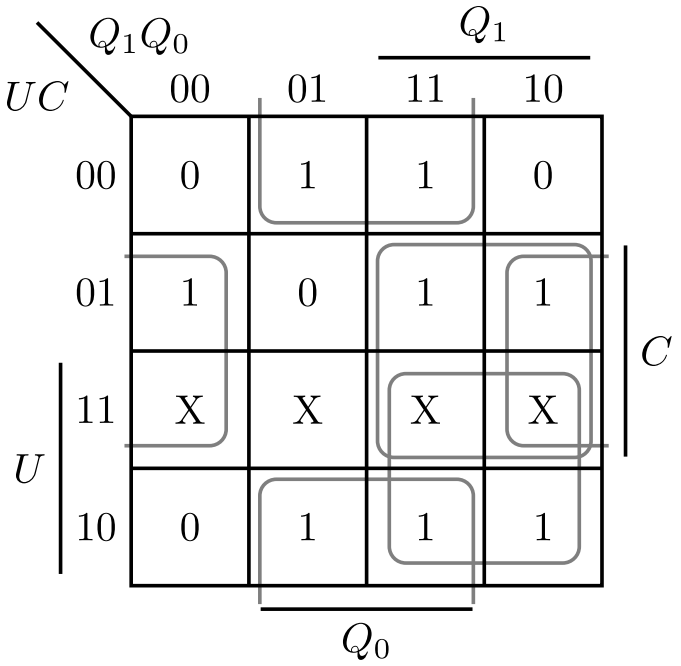
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 1: distributore automatico

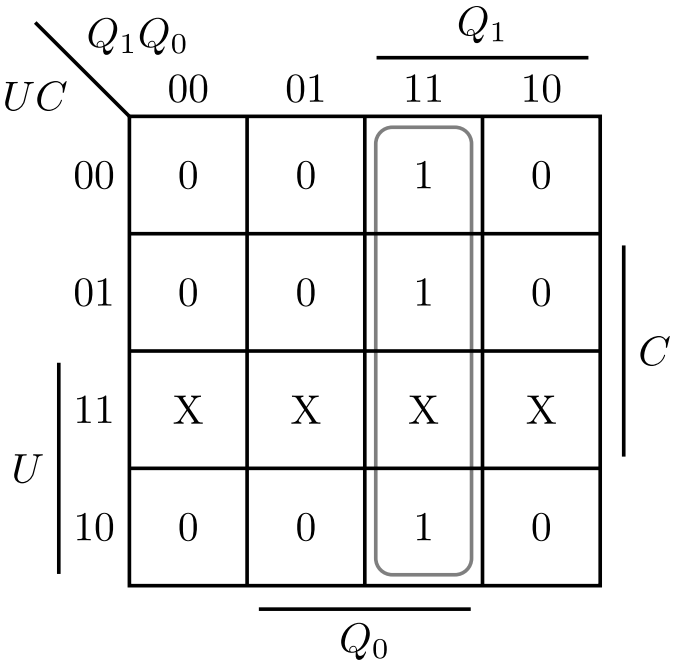
- Passo 5: scelte tecnologiche e implementative
  - A titolo di esempio, si propone una soluzione che faccia uso di flip-flop D, per semplicità



$$D_1 = Q_1 + U + Q_0C$$

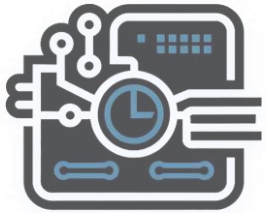


$$D_0 = Q_1U + Q_1C + Q_0\overline{C} + \overline{Q_0}C$$



$$O = Q_1Q_0$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

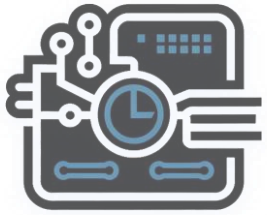


Esempio 1: distributore automatico
Riepilogo

Flusso di progetto applicato

1. *Analisi* delle specifiche
   ➔ Identificare ingressi, uscite, comportamento
2. *Definizione* FSM
   ➔ Diagramma degli stati e transizioni
3. *Minimizzazione* stati (approfondita nella lezione successiva)
   ➔ Identificare stati equivalenti
4. *Codifica* stati (approfondita nella lezione successiva)
   ➔ Tabella simbolica → tabella codificata
5. *Implementazione*
   ➔ Scelta del tipo dei flip-flop e sintesi circuito

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Due approcci fondamentali
Macchine di Mealy oppure macchine di Moore

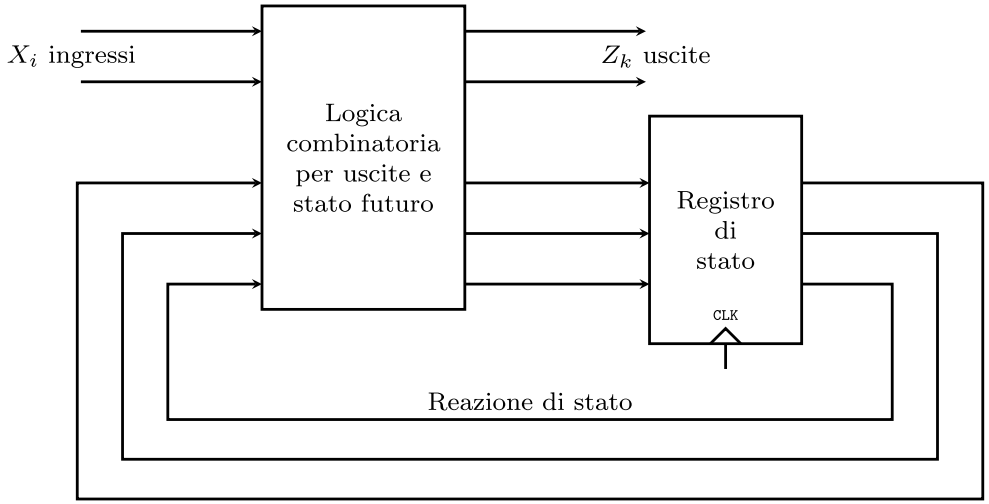
- Diverso compromesso tra
  - Numero di stati necessari
  - Complessità della logica di uscita
  - Sincronizzazione temporale
- Quando usare Mealy?
  - ✓ Uscite che devono reagire immediatamente agli ingressi
  - ✓ Riduzione del numero di stati
  - ✗ Possibili glitch sulle uscite (comportamento asincrono)
- Quando usare Moore?
  - ✓ Uscite sincronizzate con il clock (più stabili)
  - ✓ Design più pulito e predicibile
  - ✗ Richiede generalmente più stati

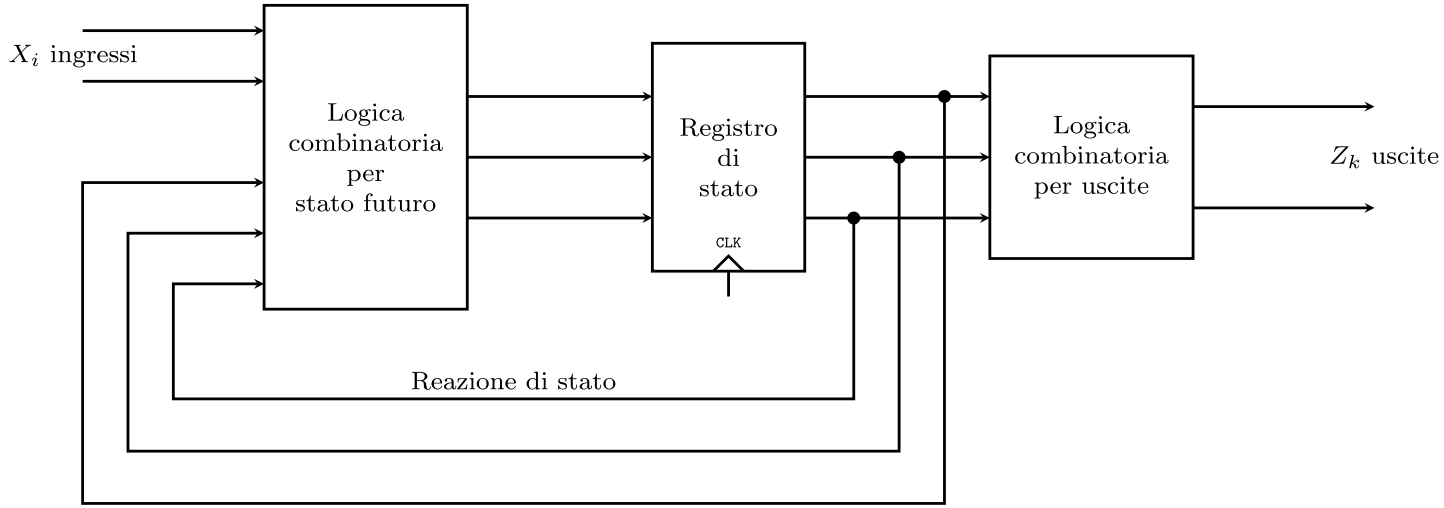
➔ *È sempre possibile convertire tra i due modelli*

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Modelli di FSM: macchine di Mealy e di Moore

- Macchina di Mealy
  - Le *uscite* dipendono da *stati ed ingressi*
  - Cambiamenti in ingresso producono *immediatamente* cambiamenti in uscita
  - Alcuni segnali presentano comportamento asincrono
- Macchina di Moore
  - Le *uscite* dipendono *solamente dagli stati*
  - I cambiamenti delle uscite sono sincronizzati con il segnale di temporizzazione

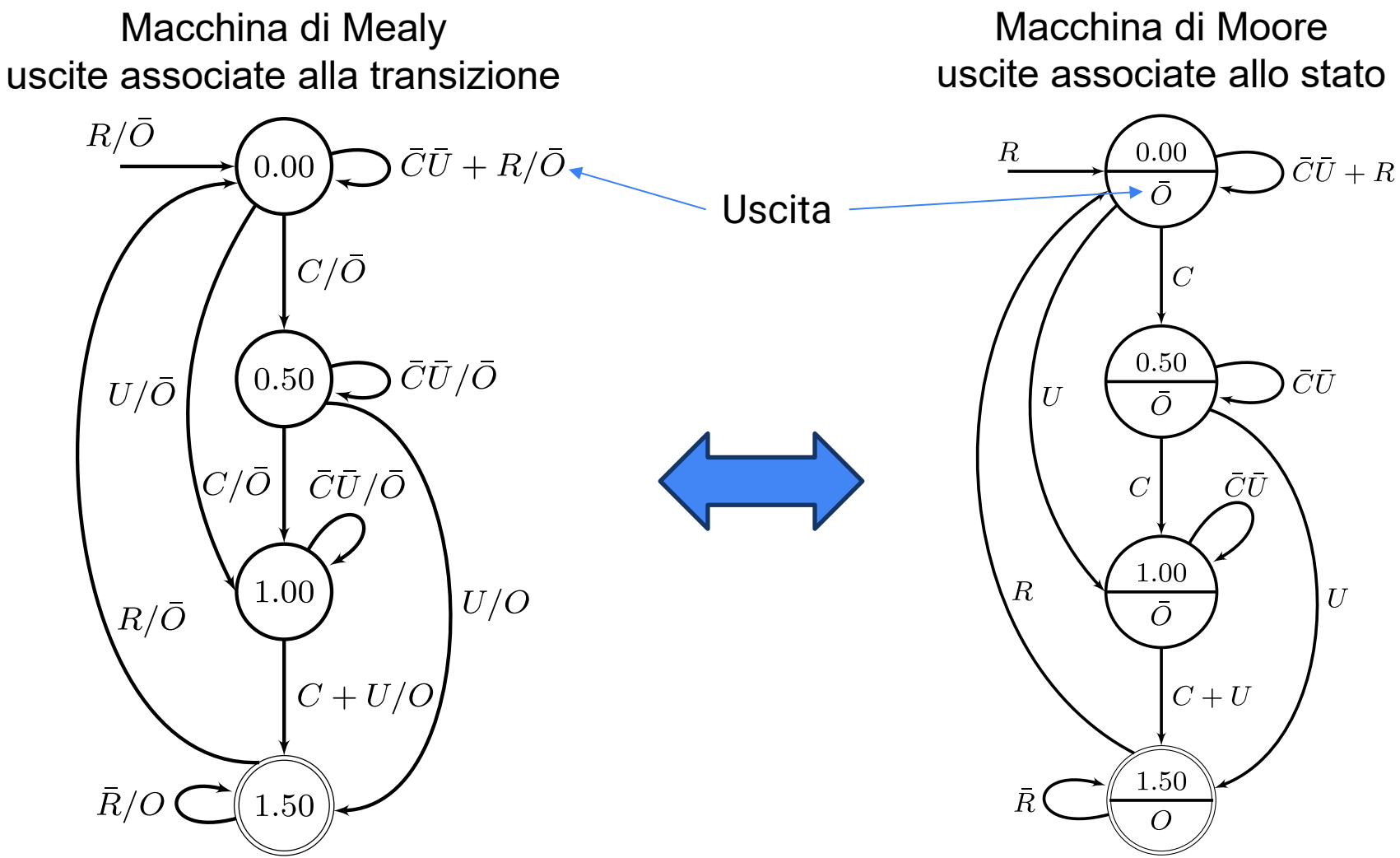




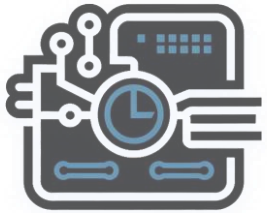
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Macchine di Mealy & Moore

- Si può sempre passare Mealy ⇔ Moore modificando i diagrammi degli stati



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

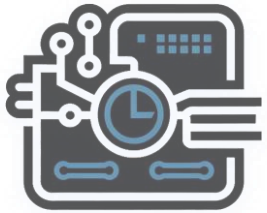


Esempio 2: riconoscitori di sequenza
Costruzione base

- Obiettivo: arrivare a un diagramma di stato di un *riconoscitore di sequenza*

1. *Partire* da uno stato iniziale in cui il riconoscimento della sequenza *non sia ancora iniziato* (stato di reset)
2. *Aggiungere* uno stato che *riconosce il primo simbolo* della sequenza
3. *Aggiungere* stati che *riconoscono i successivi simboli* della sequenza
4. Lo *stato finale* rappresenta il *riconoscimento dell'intera sequenza*, eventualmente a meno dell'ultimo simbolo

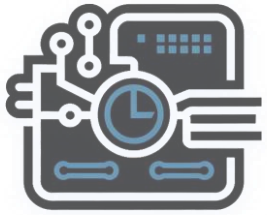
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 2: riconoscitori di sequenza
Gestione sottosequenze

5. *Aggiungere* archi di transizione di stato che specificano cosa succede quando arriva un *simbolo fuori dalla sequenza*
6. *Aggiungere* altri archi per gestire *input non nella sequenza*, che portano in stati che rappresentano *sottosequenze* di quella da riconoscere
   - Per riconoscere la sequenza *indipendentemente* da dove capiti nel flusso di simboli in ingresso

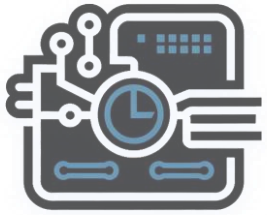
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 2: riconoscitore di sequenza «1101» Specifiche del problema (ingresso, uscita, sequenza)

- Il circuito ha come *ingresso* una linea
- Sulla linea si *trasmette una sequenza binaria*, un bit alla volta
- Si vuole riconoscere la *sequenza 1101*
- *L'uscita* vale
  - 1 solo se è stata *riconosciuta* la sequenza
  - 0 altrimenti

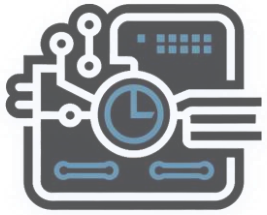
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 2: riconoscitore di sequenza «1101»
Sfida: sottosequenze sovrapposte

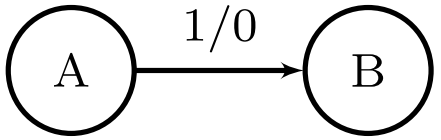
- Occorre tenere conto delle sottosequenze e di *riconoscimenti parziali*
- Esempio 1
  La sequenza *1111101* contiene *1101*,
  dove *11* è una sottosequenza propria della sequenza
  - La macchina deve *ricordare* quando riceve un altro simbolo se sono già stati ricevuti due *1*
- Esempio 2
  La sequenza *1101101* contiene *1101* sia come
  sottosequenza iniziale (*1101*101), sia come
  sottosequenza finale (*110*1101)
  con una parte di sovrapposizione
  - L'*1* nel mezzo *1101101* appartiene a *entrambe* le sequenze
  - La sequenza 1101 va riconosciuta *tutte le volte* che avviene nella sequenza in ingresso

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 2: riconoscitore di sequenza «1101»

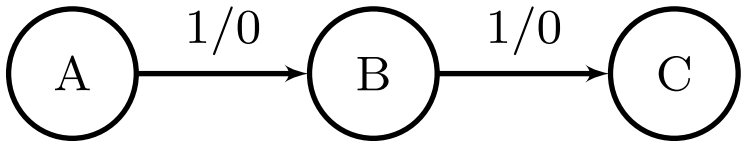
- Definizione degli stati per riconoscere la sequenza
  - Si assume che la macchina inizi il riconoscimento dal primo simbolo
  - Che continui con tutti gli altri simboli della sequenza da riconoscere
  - Che produca in uscita 1 per indicare il riconoscimento della sequenza completa, 0 altrimenti
- Si parte da uno stato iniziale A
  - Si aggiunge uno stato B che riconosce il primo *1*
    - L'uscita è sicuramente a 0 poiché non si è ancora riconosciuta la sequenza intera
- A titolo di esempio, si usa una macchina di Mealy, poi trasformata in Moore



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2: riconoscitore di sequenza «1101»

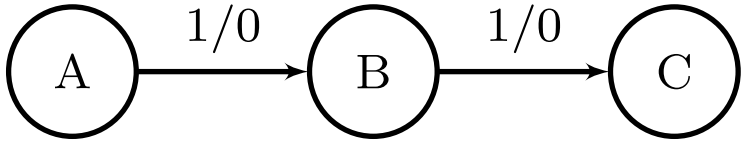
- Si introduce lo stato C che rappresenta l'aver riconosciuto una sequenza di due *1*



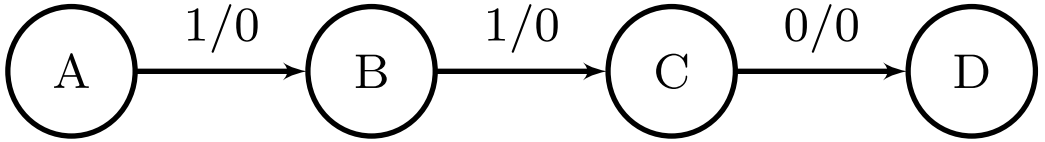
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2: riconoscitore di sequenza «1101»

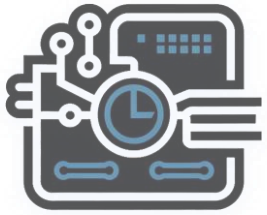
- Si introduce lo stato C che rappresenta l'aver riconosciuto una sequenza di due *1*



- Si introduce lo stato D che rappresenta l'aver riconosciuto una sequenza di due *1* seguiti da uno *0*

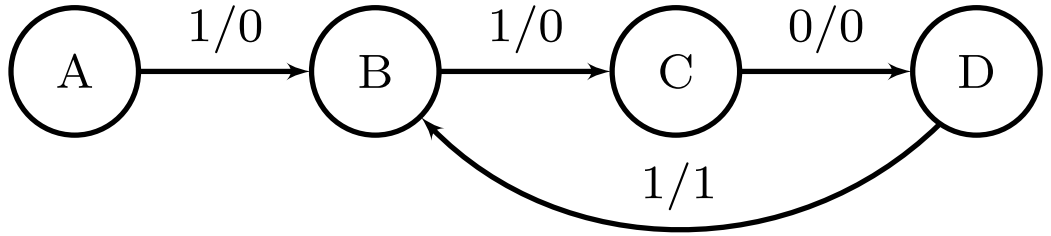


2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio 2: riconoscitore di sequenza «1101»

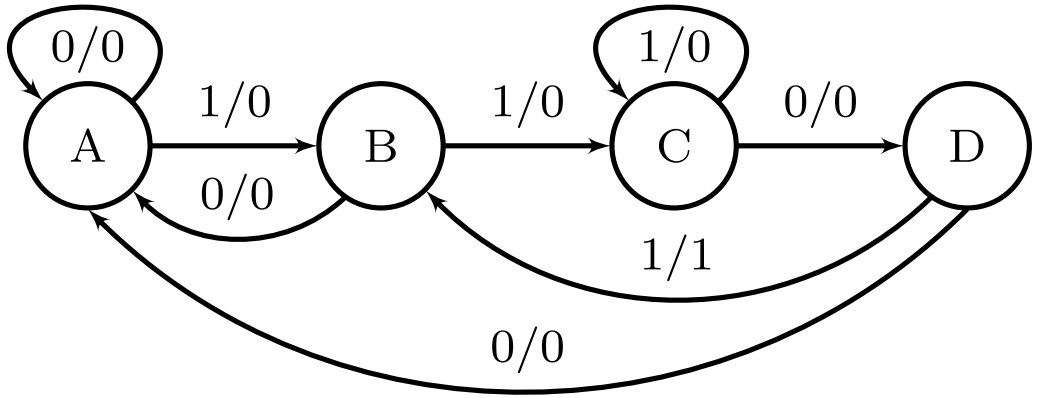
- L'arco uscente dallo stato D, a fronte di un ulteriore *1* in ingresso, deve tenere conto del fatto che la sequenza sia stata effettivamente riconosciuta
  - È la prima transizione, fino ad ora, che *asserisce l'uscita*
- Per identificare lo stato di destinazione dell'arco, va considerato che l'ultimo valore ricevuto in ingresso può far parte di una *nuova sequenza*



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 2: riconoscitore di sequenza «1101»

- Il diagramma di stato va completato con tutte le transizioni mancanti
- Si introducono tutti gli archi necessari per coprire i possibili valori di ingresso per ogni stato identificato e lo stato di destinazione corrispondente

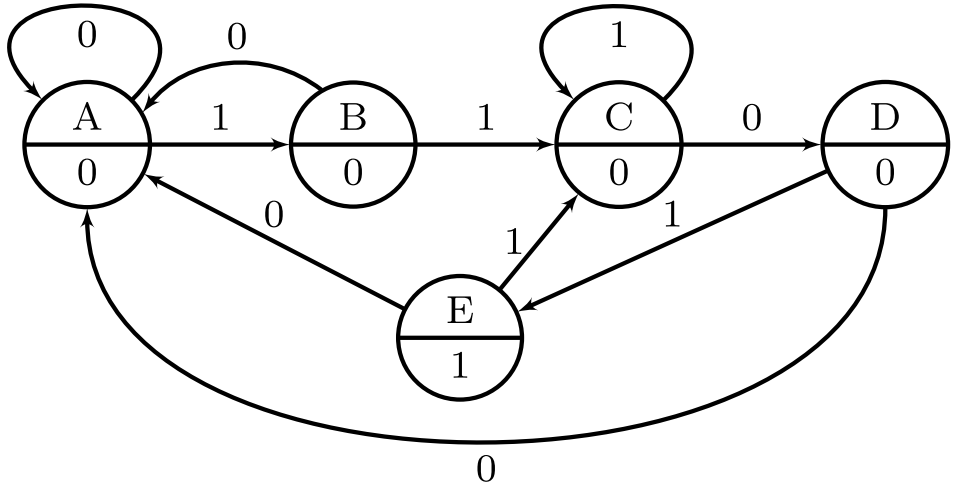


	Stato Futuro		Uscita	
Stato Presente	$x=0$	$x=1$	$x=0$	$x=1$
A	A	B	0	0
B	A	C	0	0
C	D	C	0	0
D	A	B	0	1

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

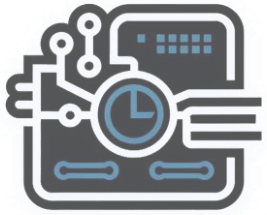
Esempio 2: riconoscitore di sequenza «1101» Conversione a macchina di Moore

- Nelle macchine di Moore, le uscite dipendono dal *solo stato*
- Serve un ulteriore stato E con uscita 1 per il valore *1* finale nella sequenza riconosciuta
  - Lo stato E, seppur simile a B, genera una *uscita diversa*, per cui i due non possono essere accorpati
  - I due stati rappresentano *astrazioni diverse* per lo storico degli ingressi
- In generale, una macchina di Moore *ha più stati* della corrispettiva macchina di Mealy



	Stato Futuro		
Stato Presente	$x=0$	$x=1$	Uscita
A	A	B	0
B	A	C	0
C	D	C	0
D	A	E	0
E	A	C	1

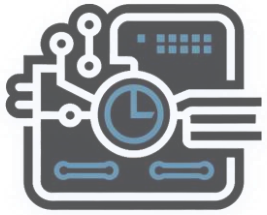
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Confronto: Mealy e Moore per il riconoscitore «1101»

	Mealy	Moore
Numero stati	**4 stati (A, B, C, D)**	**5 stati (A, B, C, D, E)**
Uscite	Dipendono da stato e ingressi	Dipendono solo dallo stato
Sincronizzazione uscite	Asincrona (immediata)	Sincrona (col clock)
Complessità logica uscite	Più compatta, ma più complessa	Più semplice, ma più stati
Ritardo uscite	**0 cicli (combinatoria)**	**1 ciclo (registrata)**

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esempio avanzato: controllore semaforico
Sistemi con temporizzatori

- Fino ad ora: FSM con ingressi puramente combinatori
- Il controllore semaforico introduce

1. *Temporizzatori*
   - Segnali che si attivano dopo intervalli di tempo
   - Richiedono stati di «attesa»
   - Timer breve (TS) per giallo
   - Timer lungo (TL) per verde
2. *Sensori esterni*
   - Rilevamento veicoli (C)
   - Condizioni variabili nel tempo
3. *Comportamento real-time*
   - Intervalli minimi e massimi
   - Priorità tra strade (principale rispetto a secondaria)
   - Sicurezza (transizioni giallo obbligatorie)

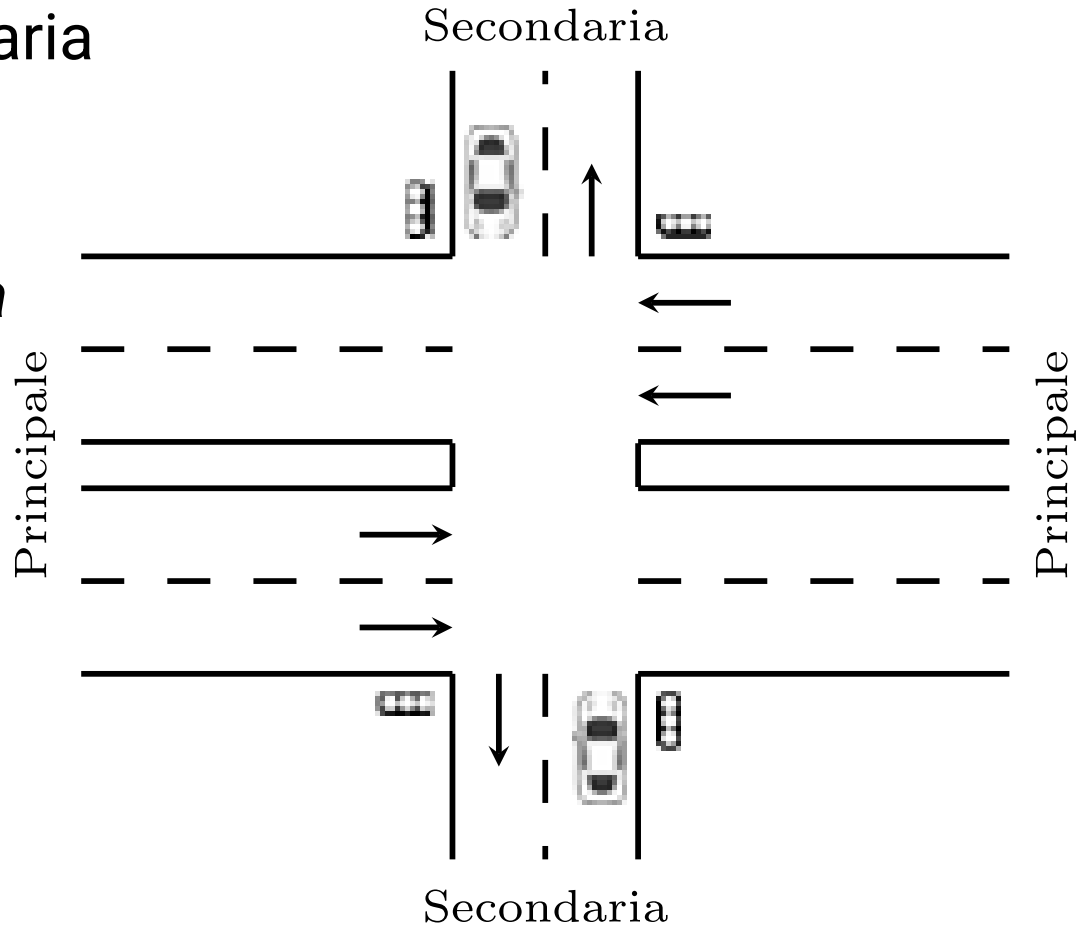
➔ *Complessità maggiore*

   - 6 stati con logica temporale

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 3: controllore semaforico
Specifica

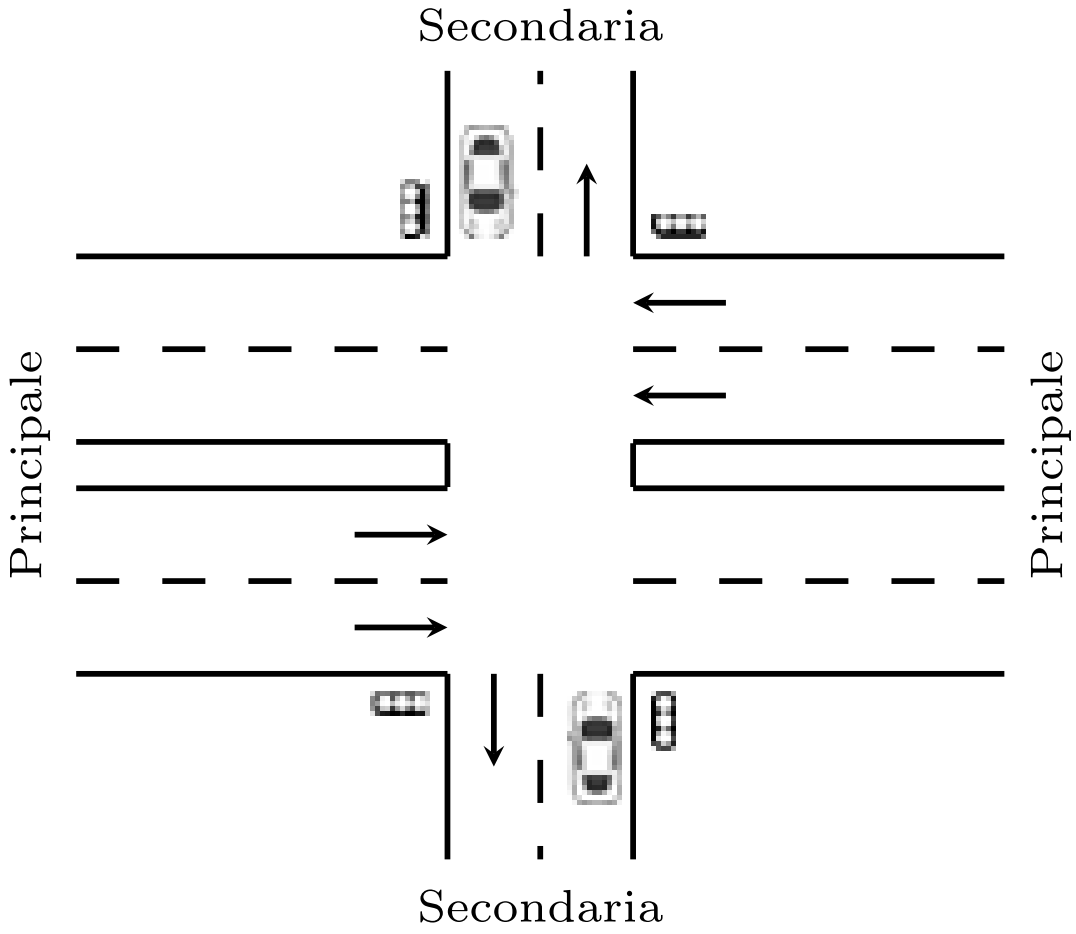
- Una strada principale incrocia una strada secondaria
- Un *sensore C* rileva auto sulla strada secondaria
- *Senza* veicoli sulla strada *secondaria*
  - Semaforo strada *principale* rimane *verde*
- Sensore *C rileva* veicolo su strada *secondaria*
  - Semaforo strada *principale* cambia:
    verde → giallo → rosso
    - Poi semaforo strada *secondaria* diventa *verde*
- Semaforo strada secondaria *rimane verde*
  - Finché ci sono veicoli, ma per un *tempo massimo*
  - Alla fine cambia: verde → giallo → rosso
- Poi semaforo strada *principale* diventa *verde*
  - Resta verde per un *intervallo minimo* prefissato
    - Anche se il *sensore C* rileva veicoli sulla strada *secondaria*



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esempio 3: controllore semaforico
Temporizzatori

- Si suppone l'esistenza di un *temporizzatore*
- *Ingresso*
  - Un segnale di *start* ST
- Uscite
  - TS dopo un *breve* intervallo di tempo
    → Si usa per *temporizzare il giallo*
  - TL dopo un *lungo* intervallo di tempo
    → Si usa per *temporizzare il verde*



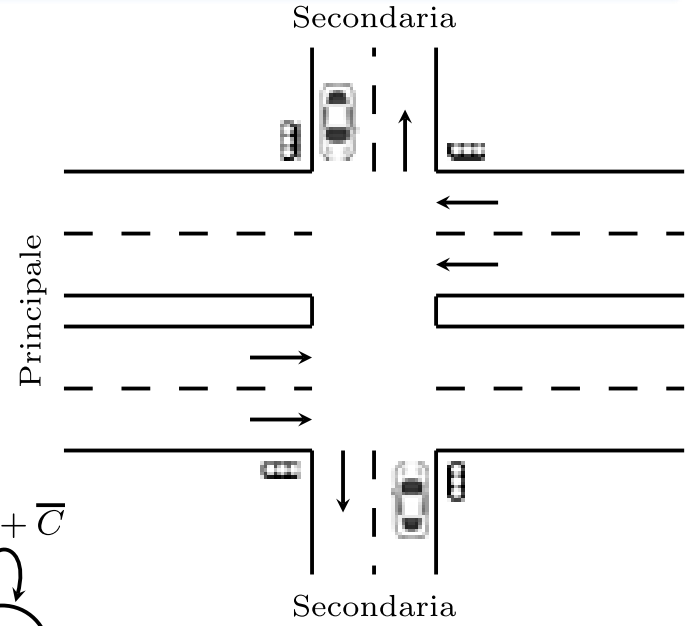
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

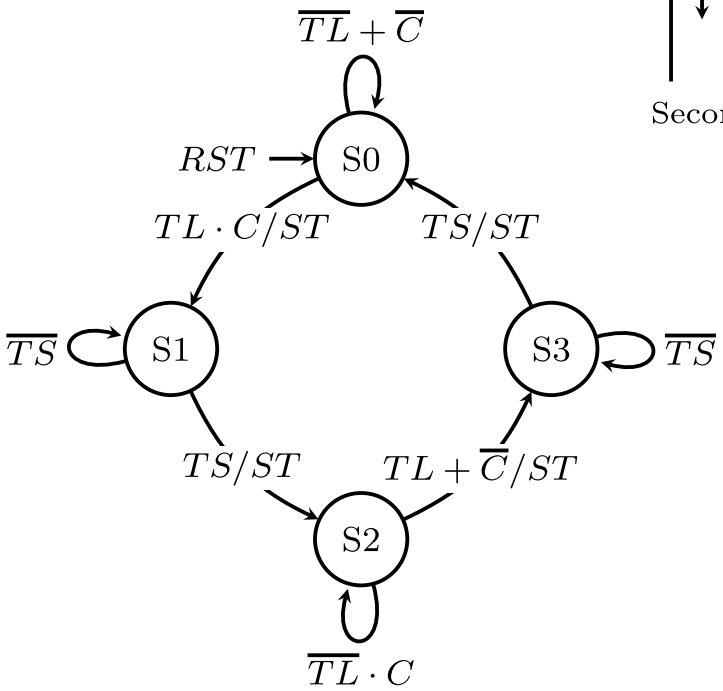
Esempio 3: controllore semaforico

Segnale di ingresso	Descrizione
RST	Porta la FSM allo stato iniziale
C	Presenza di veicolo su strada secondaria
TS	Fine intervallo breve
TL	Fine intervallo lungo

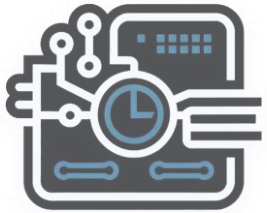
Segnale di uscita	Descrizione
HG, HY, HR	Attiva luce verde/gialla/rossa principale
FG, FY, FR	Attiva luce verde/gialla/rossa secondaria
ST	Inizio intervallo di tempo

Stato	Descrizione
S_0	Verde principale (Rosso secondaria)
S_1	Giallo principale (Rosso secondaria)
S_2	Verde secondaria (Rosso principale)
S_3	Giallo secondaria (Rosso principale)





2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



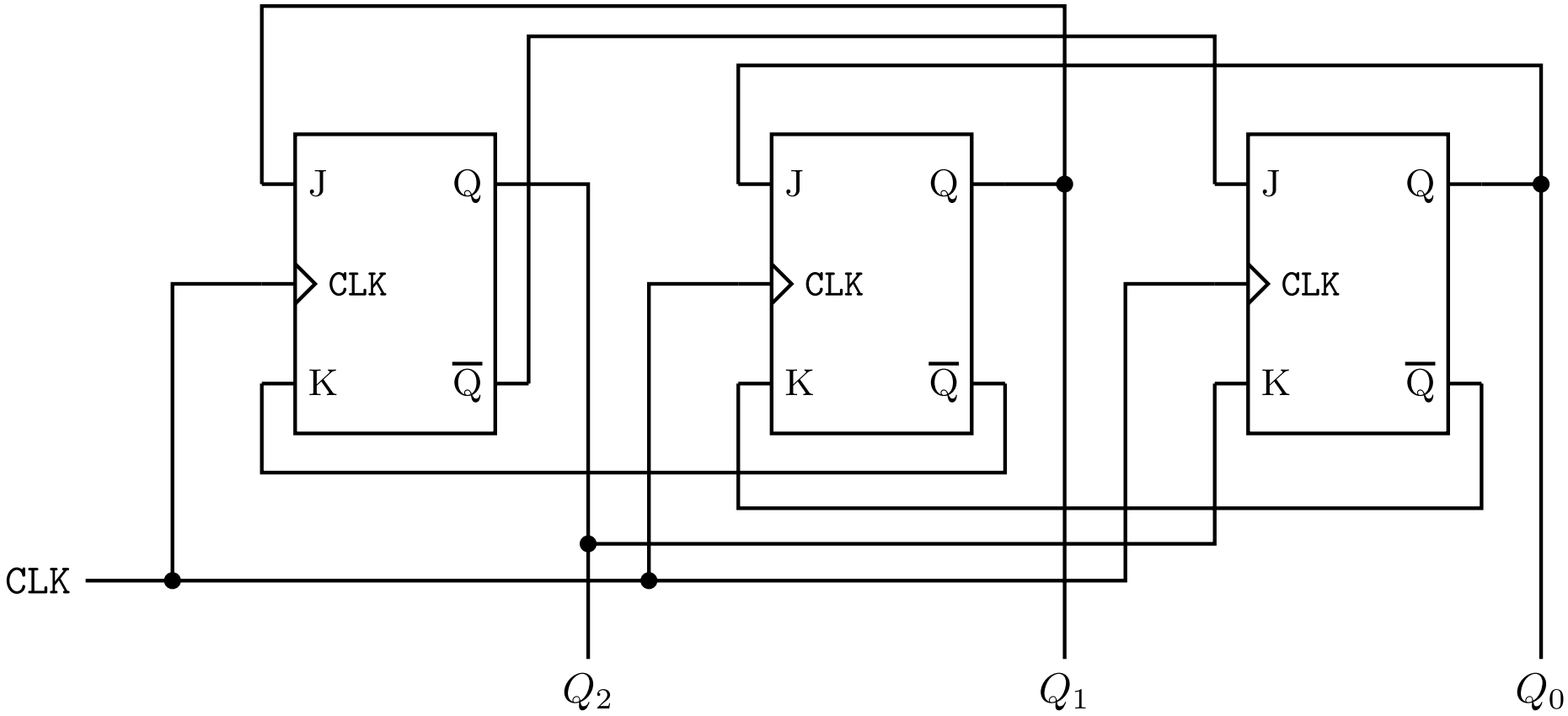
Esercitazioni guidate

- Applicazione pratica dei concetti visti
- Esercizio 5.1 → analisi di circuito sequenziale
  - *Dato*: circuito con flip-flop JK
  - *Obiettivo*: derivare il diagramma degli stati
  - *Percorso*: schema → mappe Karnaugh → equazioni → grafo
- Esercizio 5.2 → design da specifica
  - *Dato*: specifica comportamentale (sequenza "101")
  - *Obiettivo*: progettare il circuito completo
  - *Percorso*: specifica → FSM → mappe Karnaugh → circuito
- Il flusso di progetto sarà seguito
  → In avanti
  → All'indietro

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)

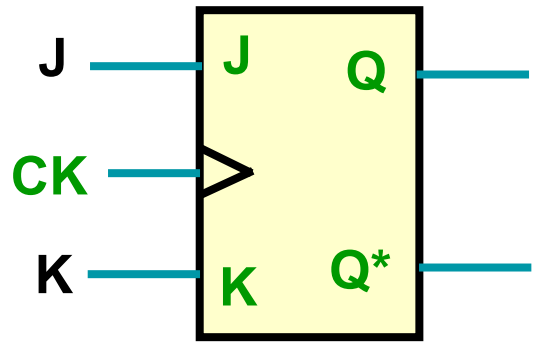
- Dato un *circuito sequenziale*, determinare il grafo di *transizione degli stati* usando le corrispondenti mappe di Karnaugh



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)
Passo 1: tabella di verità di un flip-flop JK

- I *dispositivi di memoria* nel circuito sequenziale sono flip-flop JK → funzionano secondo la loro *tabella di verità*



J	K	Q	Q*
0	0	Q_{-1}	Q^*_{-1}
0	1	0	1
1	0	1	0
1	1	Q_{-1}*	Q_{-1}

Condizione di memoria

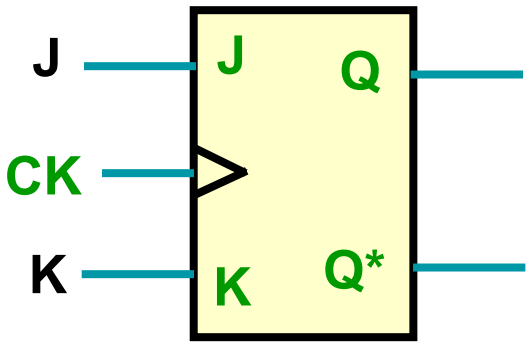
per $J, K = 1,1$ l'uscita commuta a ogni colpo di clock (toggle)

Condizioni di comando dell'uscita

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)
Passo 2: mappa di Karnaugh per un flip-flop JK

- Dalla tabella di verità del flip-flop JK si ottiene la sua *funzione caratteristica*, minimizzata attraverso la mappa di Karnaugh



J	K	Q	Q*
0	0	Q_{-1}	Q^*_{-1}
0	1	0	1
1	0	1	0
1	1	Q_{-1}*	Q_{-1}

Condizione di memoria

per J,K = 1,1 l'uscita commuta a ogni colpo di clock (toggle)

Condizioni di comando dell'uscita

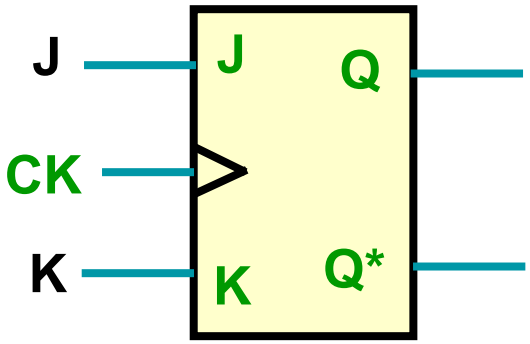
Q^+

Q \ JK	00	01	11	10
0	0	0	1	1
1	1	0	0	1

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)
Passo 3: funzione caratteristica dell'uscita di un flip-flop JK

- Dalla tabella di verità del flip-flop JK si ottiene la sua *funzione caratteristica*, minimizzata attraverso la mappa di Karnaugh



J	K	Q	Q*
0	0	Q_{-1}	Q^*_{-1}
0	1	0	1
1	0	1	0
1	1	Q_{-1}*	Q_{-1}

Condizione di memoria

per J,K = 1,1 l'uscita commuta a ogni colpo di clock (toggle)

Condizioni di comando dell'uscita

Q^+

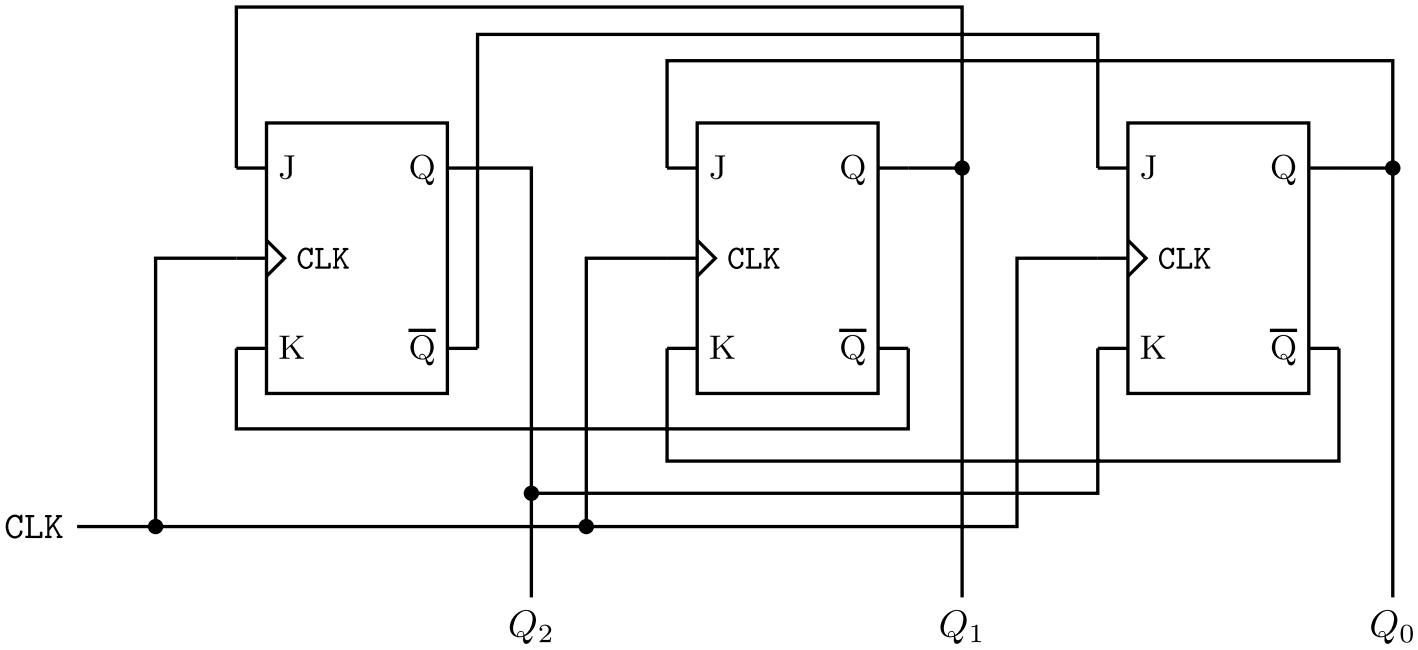
Q \ JK	00	01	11	10
0	0	0	1	1
1	1	0	0	1

$$Q^+ = J\bar{Q} + Q\bar{K}$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)
Passo 4: equazioni stato futuro

- Le funzioni caratteristiche dei flip-flop JK determinano le *equazioni dello stato futuro* delle uscite, quindi i *cambiamenti di stato* della FSM



$$Q^+ = J\,\overline{Q} + Q\,\overline{K} \Longrightarrow \begin{cases} Q_2^+ = Q_1\,\overline{Q_2} + Q_2\,\overline{\overline{Q_1}} \\ Q_1^+ = Q_0\,\overline{Q_1} + Q_1\,\overline{\overline{Q_0}} \\ Q_0^+ = \overline{Q_2}\,\overline{Q_0} + Q_0\,\overline{Q_2} \end{cases}$$

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)
Passo 5: sequenza transizioni

- Si completa la mappa di Karnaugh per l'intero circuito sequenziale (FSM) da cui si ricava la *sequenza di transizioni di stato*

$$\begin{cases} Q_2^+ = Q_1\,\overline{Q_2} + Q_2\,Q_1 \\ Q_1^+ = Q_0\,\overline{Q_1} + Q_1\,Q_0 \\ Q_0^+ = \overline{Q_2}\,\overline{Q_0} + Q_0\,\overline{Q_2} \end{cases}\Bigg|_{Q_2=0,Q_1=0,Q_0=0} \Rightarrow \begin{cases} Q_2^+ = 0\cdot 1 + 0\cdot 0 = 0 \\ Q_1^+ = 0\cdot 1 + 0\cdot 0 = 0 \\ Q_0^+ = 1\cdot 1 + 0\cdot 1 = 1 \end{cases}$$

$$\begin{cases} Q_2^+ = Q_1\,\overline{Q_2} + Q_2\,Q_1 \\ Q_1^+ = Q_0\,\overline{Q_1} + Q_1\,Q_0 \\ Q_0^+ = \overline{Q_2}\,\overline{Q_0} + Q_0\,\overline{Q_2} \end{cases}\Bigg|_{Q_2=0,Q_1=0,Q_0=1} \Rightarrow \begin{cases} Q_2^+ = 0\cdot 1 + 0\cdot 0 = 0 \\ Q_1^+ = 1\cdot 1 + 0\cdot 1 = 1 \\ Q_0^+ = 1\cdot 0 + 1\cdot 1 = 1 \end{cases}$$

$$\begin{cases} Q_2^+ = Q_1\,\overline{Q_2} + Q_2\,Q_1 \\ Q_1^+ = Q_0\,\overline{Q_1} + Q_1\,Q_0 \\ Q_0^+ = \overline{Q_2}\,\overline{Q_0} + Q_0\,\overline{Q_2} \end{cases}\Bigg|_{Q_2=0,Q_1=1,Q_0=1} \Rightarrow \begin{cases} Q_2^+ = 1\cdot 1 + 0\cdot 1 = 1 \\ Q_1^+ = 1\cdot 0 + 1\cdot 1 = 1 \\ Q_0^+ = 1\cdot 0 + 1\cdot 1 = 1 \end{cases}$$

. . .

$Q_2Q_1Q_0$	Q_2Q_1 \ Q_2	00	01	11	10
Q_0	0	001	101	100	000
Q_0	1	011	111	110	010

Q_1

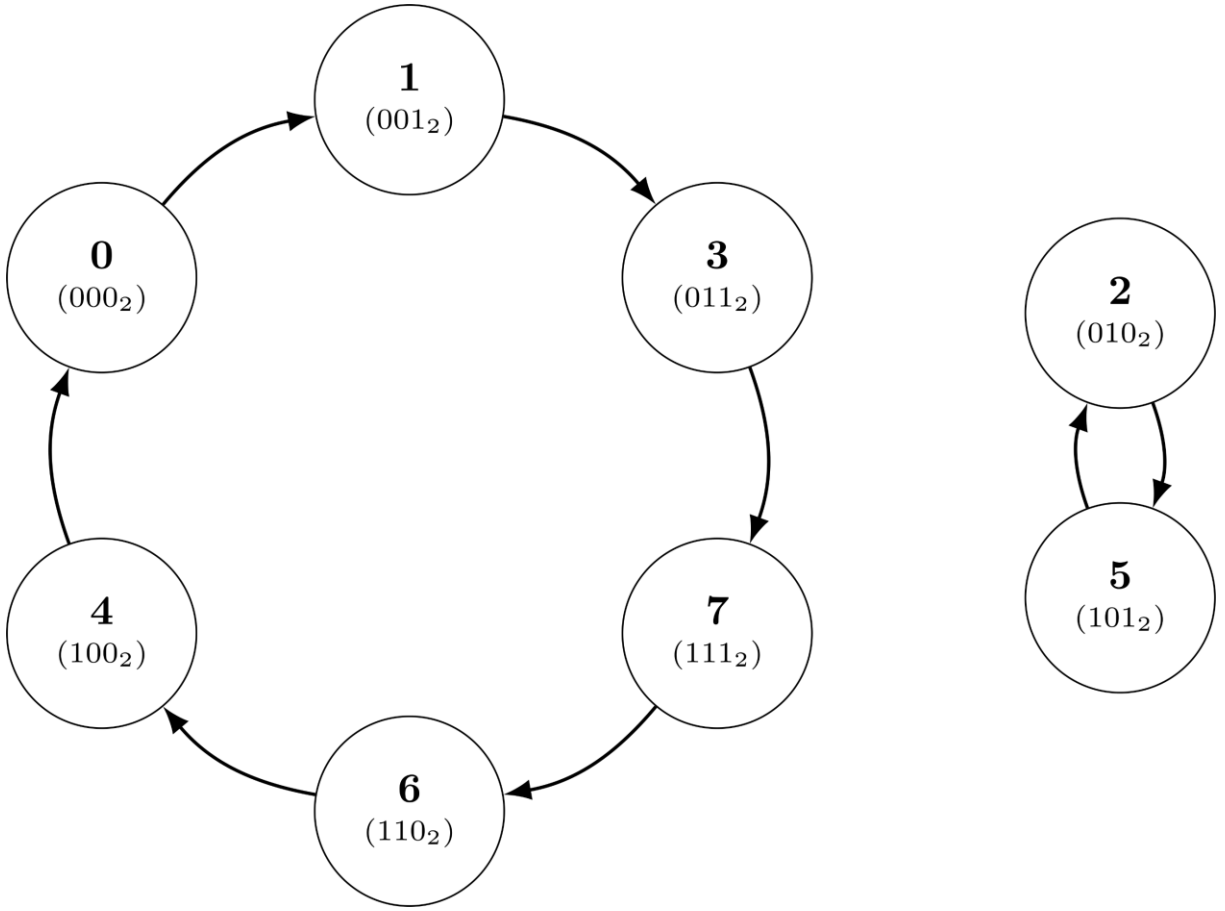
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.1: analisi di un circuito sequenziale (FSM)
Passo 5: sequenza transizioni

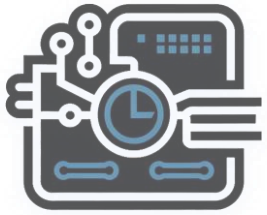
- Si completa la mappa di Karnaugh per l'intero circuito sequenziale (FSM) da cui si ricava la *sequenza di transizioni di stato*

$Q_2Q_1Q_0$	Q_2Q_1 = 00	01	Q_2 = 11	10
Q_0 = 0	001	101	100	000
Q_0 = 1	011	111	110	010

(Q_1)



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Esercizio 5.2: design di circuito sequenziale da specifica

- Data la seguente specifica del comportamento atteso, identificare un circuito in grado di implementare la funzionalità richiesta
  - Modellare un riconoscitore di sequenza adatto a riconoscere una *qualsiasi* sequenza che termini in «*101*»

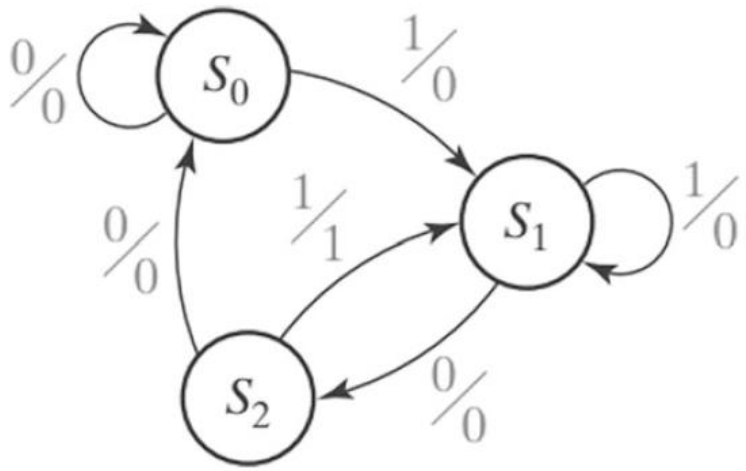
Step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
X	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0	…
Z	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	…

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.2: design di circuito sequenziale da specifica

- Modellare un riconoscitore di sequenza addato a riconoscere una qualsiasi sequenza che termini in «*101*»
  - FSM modellata come *transizioni di stato* per architettura di Mealy
  - → Coppie X/Z come etichetta sugli archi

Step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
X	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0	…
Z	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	…



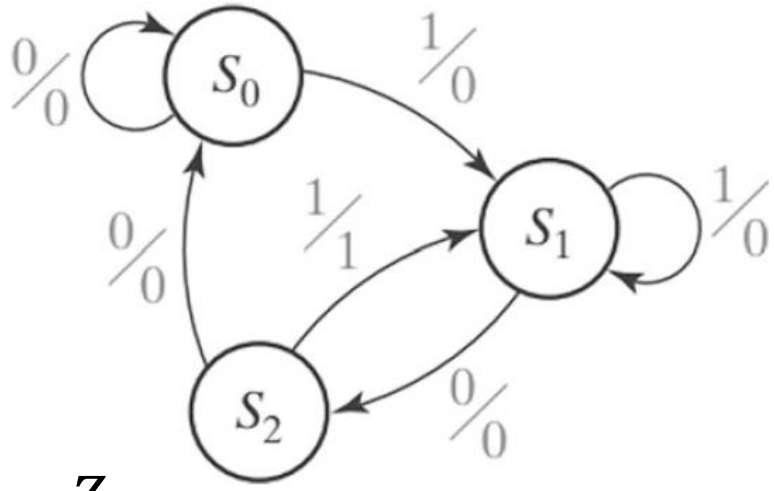
2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.2: design di circuito sequenziale da specifica

- A partire dalle transizioni di stato della FSM identificata, si definiscono le *mappe di Karnaugh* per lo stato futuro dei FF (Q_1^+, Q_2^+) e dell'uscita Z

$Q_1^+ Q_2^+ Z$

X \ Q_1Q_2	S_0 00	S_1 01	11	S_2 10
0	000	100	×	000
1	010	010	×	011



Q_1^+

X \ Q_1Q_2	00	01	11	10
0	0	1	×	0
1	0	0	×	0

Q_2^+

X \ Q_1Q_2	00	01	11	10
0	0	0	×	0
1	1	1	×	1

Z

X \ Q_1Q_2	00	01	11	10
0	0	0	×	0
1	0	0	×	1

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Esercizio 5.2: design di circuito sequenziale da specifica Implementazione

- Utilizzando dei flip-flop tipo D

$D_{Q_1} = Q_1^+$

X \ Q_1Q_2	00	01	11	10
0	0	1	×	0
1	0	0	×	0

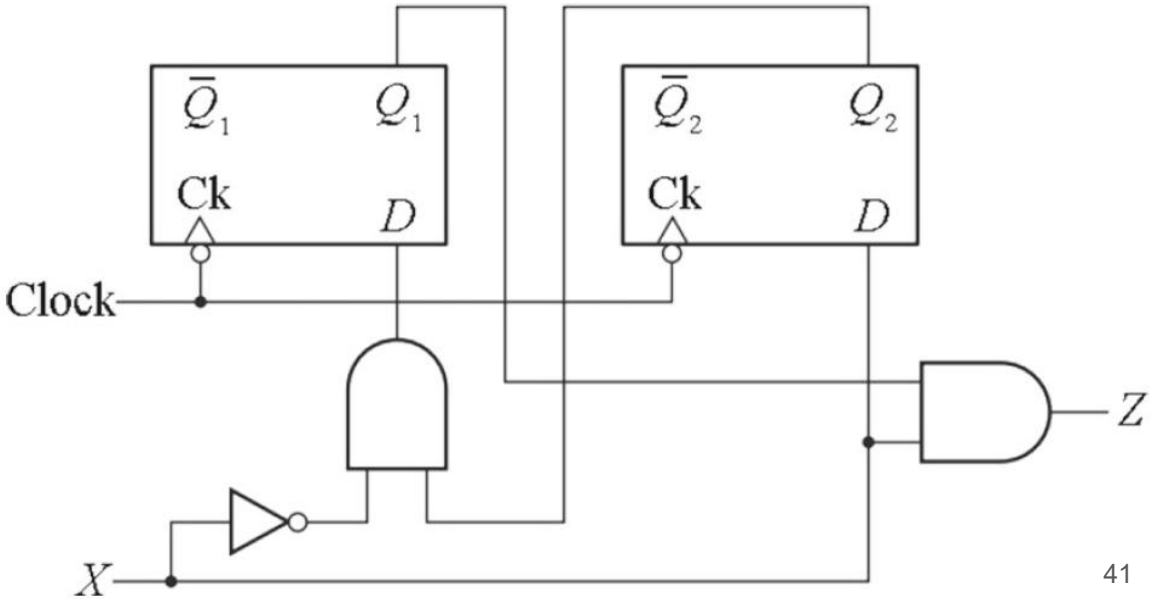
$D_{Q_2} = Q_2^+$

X \ Q_1Q_2	00	01	11	10
0	0	0	×	0
1	1	1	×	1

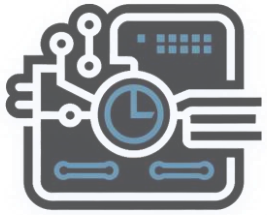
Z

X \ Q_1Q_2	00	01	11	10
0	0	0	×	0
1	0	0	×	1

$$\begin{cases} D_{Q_1} = Q_1^+ = \bar{X}\,Q_2, \\ D_{Q_2} = Q_2^+ = X, \\ Z = X\,Q_1, \end{cases}$$



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Argomenti di approfondimento individuale

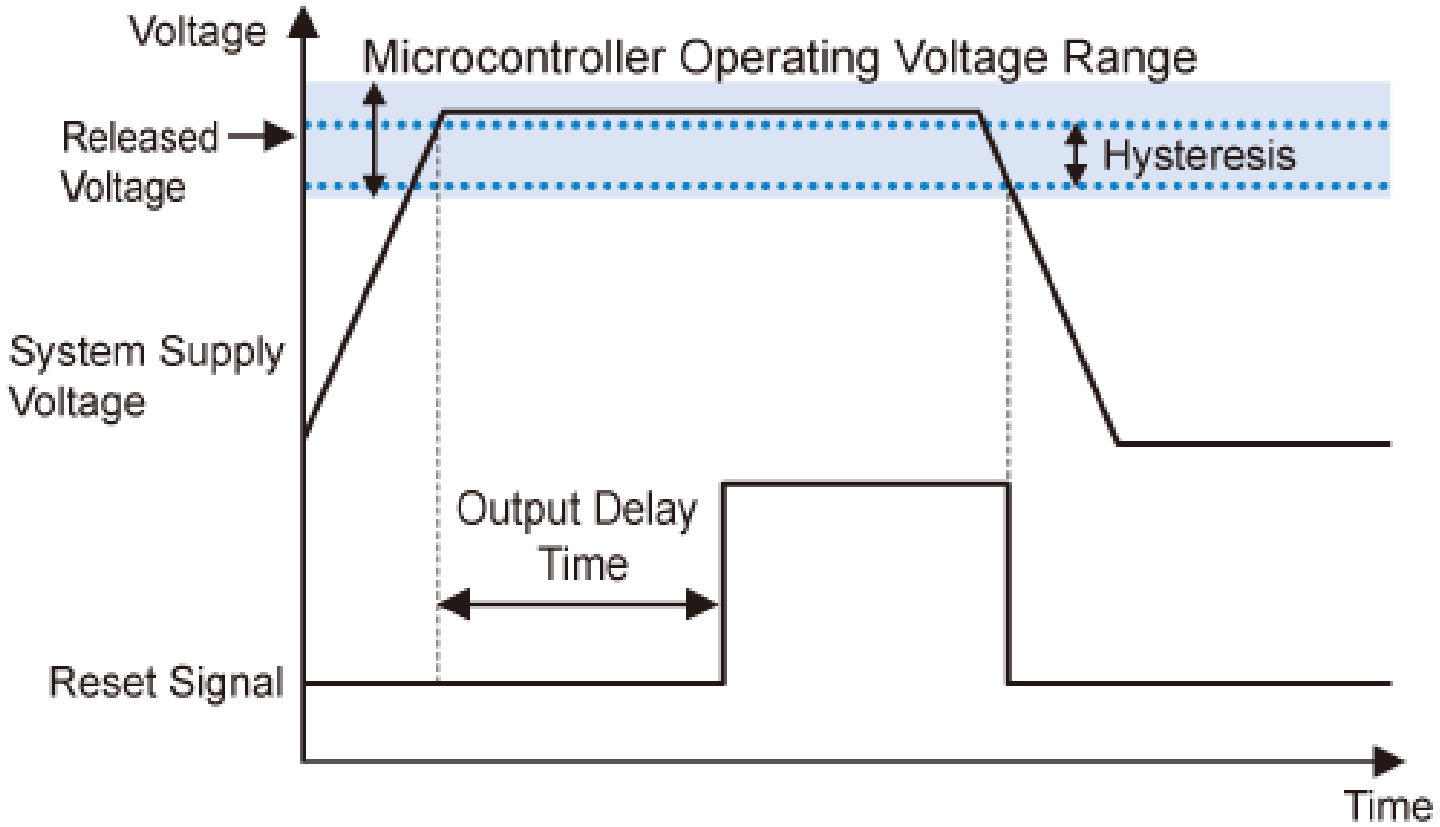
- A1) Power on Reset

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Power-on Reset (PoR) Introduzione

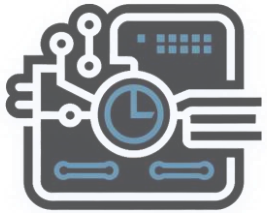
A1

- Power-on reset (PoR) → inizializzazione corretta dei sistemi digitali all'accensione
  - Il sistema deve sempre partire in uno stato predefinito
  - Fondamentale per affidabilità e prevedibilità
  - Essenziale praticamente in tutti i sistemi digitali moderni



© Nisshinbo Micro Devices Inc. What is a power on reset? (2025).

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini



Power-on Reset (PoR)
Necessità

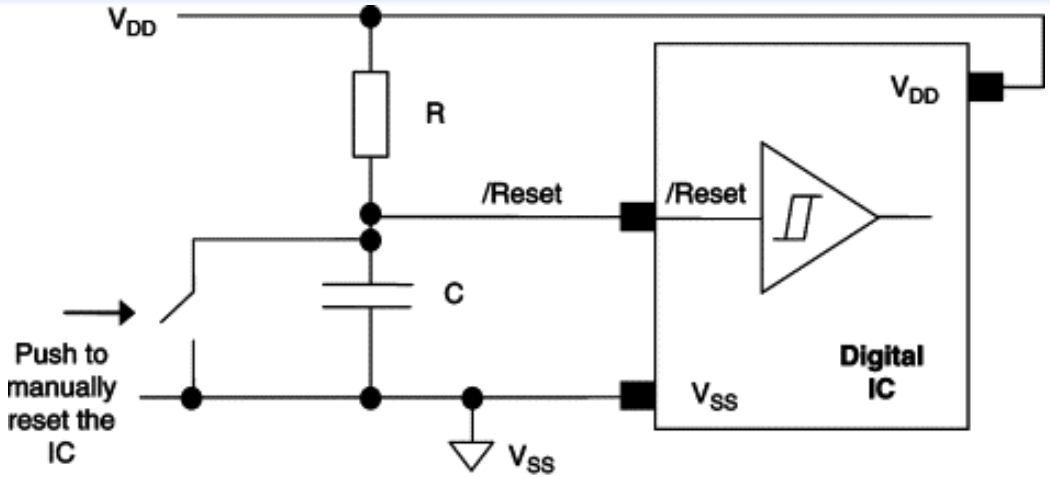
- I circuiti digitali si possono avviare in stati imprevedibili
- L'alimentazione non raggiunge istantaneamente il valore nominale
  - Rampa di tensione di valori intermedi, con tempistiche variabili
- Componenti diversi si stabilizzano con tempistiche differenti
- Conseguenze senza PoR per i sistemi digitali
  - Comportamenti anomali e imprevedibili
  - Possibili danni ai componenti
  - Corruzione dei dati
  - Malfunzionamenti sistematici all'avvio

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

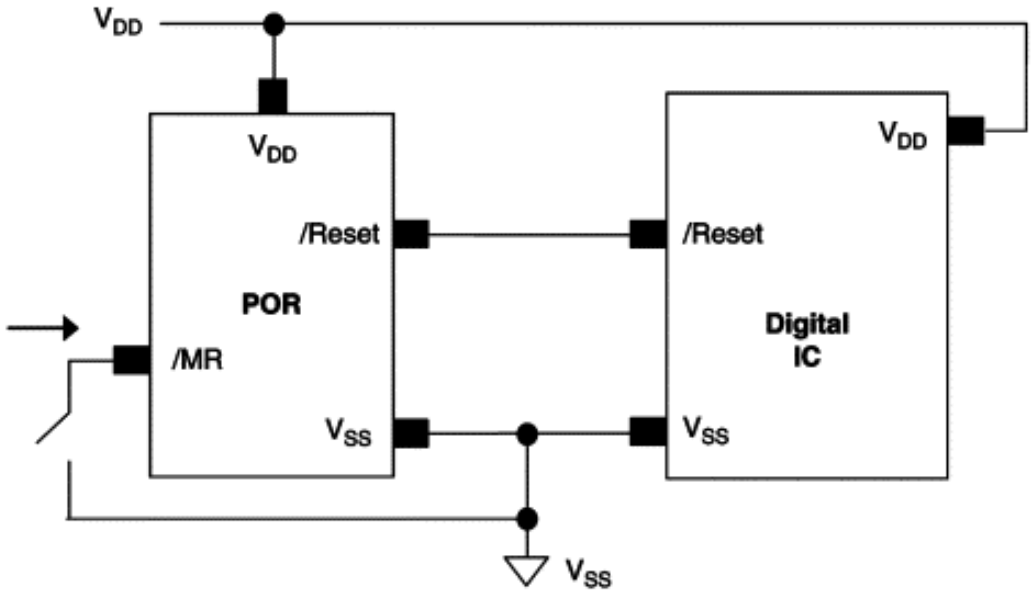
Power-on Reset (PoR) Funzionamento

© Ian Grout, Ch. 2 - Electronic Systems Design. Digital Systems Design with FPGAs and CPLDs. Newnes, 2008, Pages 43-121.

- Componenti fondamentali
  - Circuito RC: crea un ritardo temporale per attendere la stabilizzazione della tensione
  - Trigger di Schmitt: fornisce isteresi per evitare oscillazioni durante la transizione
  - Supervisore di tensione: monitora quando l'alimentazione raggiunge livelli sicuri
- Principio di funzionamento
  - Il segnale di reset rimane attivo finché la tensione non supera la soglia predefinita
  - Il circuito RC determina la temporizzazione del rilascio del reset
- Varianti realizzative
  - Circuiti integrati dedicati
  - Soluzioni con componenti discreti
  - PoR integrato nei microcontrollori



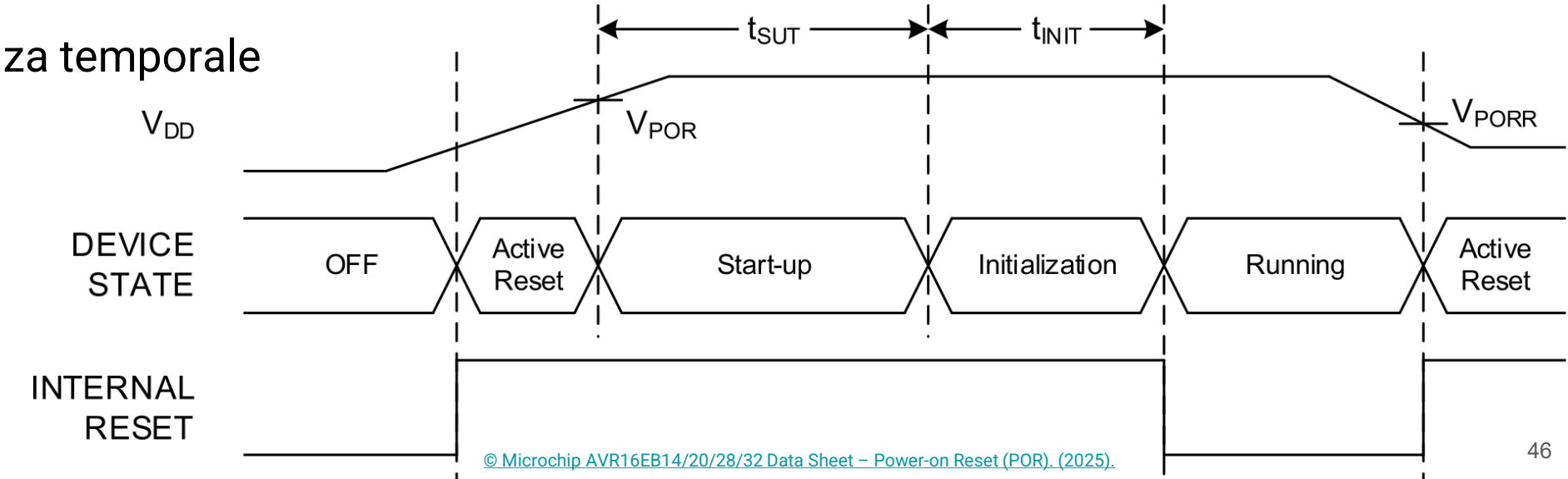
(a) Discrete RC network



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Power-on Reset (PoR)
Sequenza di inizializzazione

- Sequenza temporale
  1. Pulsante di accensione → attivazione alimentatore
  2. Rampa di tensione sui rail di alimentazione
  3. Monitoraggio dei livelli di tensione da parte del PoR
  4. Generazione del segnale "Power Good" quando tutte le tensioni sono stabili
  5. Rilascio del reset della CPU/controller
  6. Inizializzazione di registri, memoria e periferiche
  7. La CPU inizia l'esecuzione dall'indirizzo di reset
- Elementi critici
  - Corretta sequenza temporale
  - Stabilità segnali
  - Inizializzazione ordinata dei sottosistemi

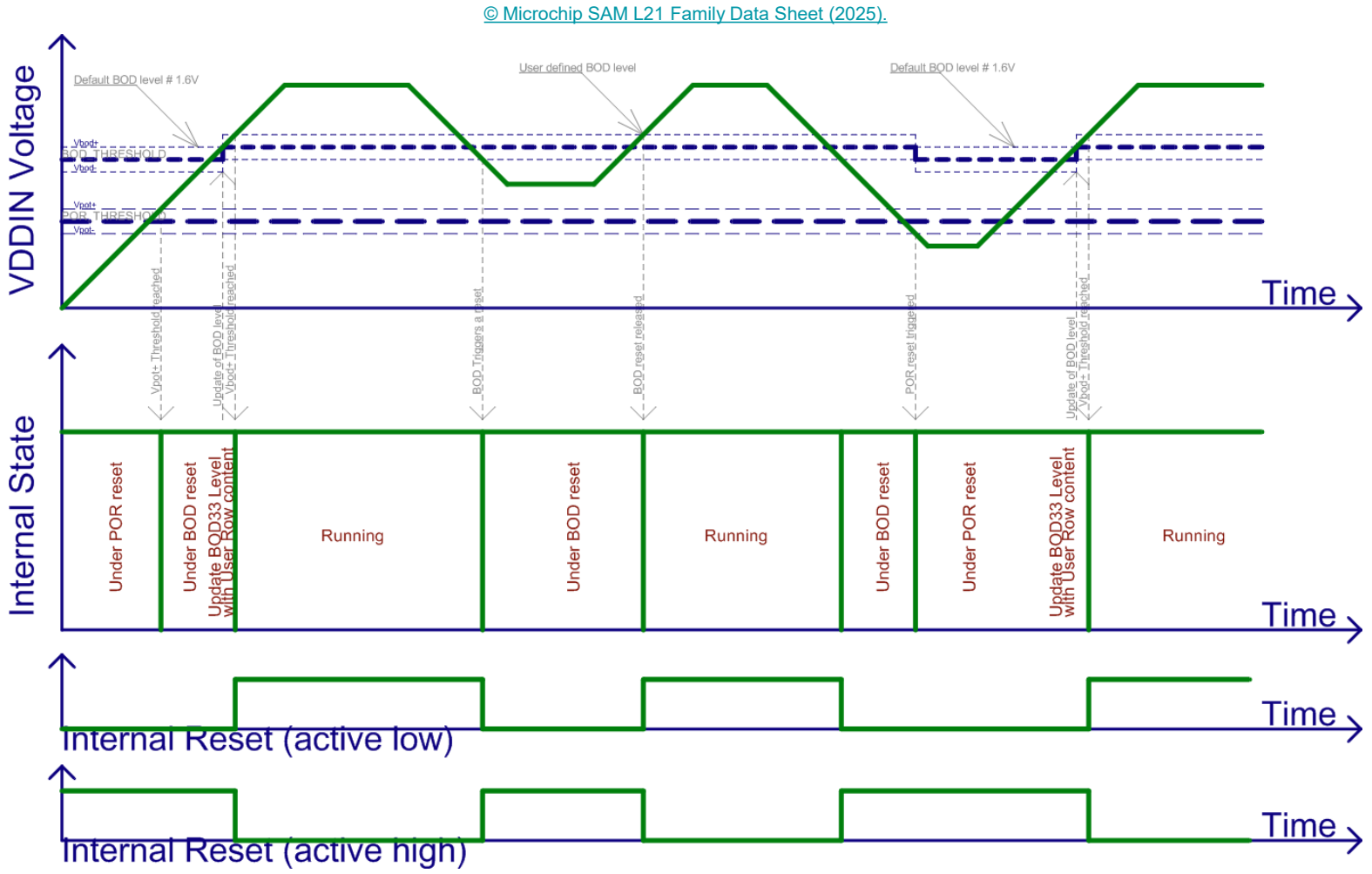


© Microchip AVR16EB14/20/28/32 Data Sheet – Power-on Reset (POR). (2025).

2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini

Power-on Reset (PoR)
Caratteristiche avanzate e applicazioni

- Funzionalità avanzate
  - Rilevamento brownout (cali temporanei di tensione)
  - Soglie di tensione regolabili
  - Reset manuale
  - Watchdog integrato (monitoraggio continuo)
- Esempi applicativi
  - Sistemi embedded (IoT, automotive, industriale)
  - Microcontrollori e DSP
  - FPGA e ASIC
  - Sistemi mission-critical
- Considerazioni di design
  - Dimensionamento temporizzazione (circuito RC) in base ai tempi di stabilizzazione
  - Immunità al rumore, consumo energetico, affidabilità nel tempo



2024–2026 © Mario Casu, Mihai Lazarescu, Paolo Pasini